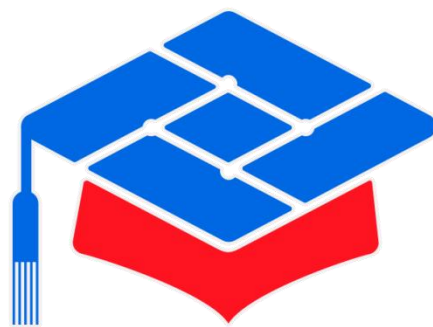




第十届 全国大学生集成电路创新创业大赛

报告类型： 技术文档报告

参赛杯赛： “紫光同创”企业命题

作品名称： 基于紫光同创 FPGA 的智能交通感知系统

队伍编号： 1007593

团队名称： Elephant-大象

重要内容预览页

本项目面向嵌入式智能交通场景，基于紫光同创 Logos 系列国产 FPGA 芯片，完成图像采集 - 预处理 - 车牌定位识别 - 结果叠加显示全流程纯硬件化实现，完全符合赛事命题设计要求，经多场景实测验证，核心技术亮点与实测性能指标如下：

1. 全硬件纯逻辑流水线加速，实时性高

核心算法全流程采用 Verilog 纯逻辑实现，无 RISC-V/ARM 软核依赖，构建全流水线并行处理架构，支持 1920×1080@60fps 高清视频流实时处理，单帧算法处理延迟 < 1ms，端到端全链路延迟≤166.3ms，上位机帧率达到 5 帧；对比同场景下纯 ARM 方案平均 2 帧 / 秒、单帧识别延迟≈516.6ms 的性能表现，实时性大幅提升，完全满足智能交通场景的感知需求

2. FPGA 适配，ARM 资源占用优化

FPGA 核心逻辑资源占用极低：LUT 占用仅 14%、FF 占用仅 8%、BRAM 占用远低于 15%，预留超 70% 的逻辑资源余量与完整的外设拓展能力；
ARM 上位机资源占用极少，四核占用率均未超过 30%，且部署于树莓派这样算力较低的开发板，有极强的可移植性和性价比，符合轻量化嵌入式识别车牌的场景，在极致的功耗和低资源下可以实现优秀的识别正确率

3. 算法创新，全类型车牌兼容

严格遵循《中华人民共和国机动车号牌》(GA 36-2018) 标准，创新采用交叉乘法无除法 RGB 比例颜色分割算法，完全规避 FPGA 中高资源占用的除法器与复杂 HSV 色域转换，原生覆盖全类型车牌检测。经实测，针对特殊车牌场景：
黑色警用车牌识别准确率达 89%（105/117）、
末尾汉字特殊车牌识别准确率达 81%（40/49）、
拼色特殊车牌识别准确率达 54%（63/115）， 实现民用常规车牌与特殊行业车牌的兼容。

4. 多级鲁棒性设计，复杂场景实测准确率高

针对强光、逆光、夜间、倾斜、远距离等复杂交通场景，构建多级硬件化鲁棒性算法架构：
集成 Gamma 校正、3x3 高斯滤波降噪、形态学闭运算、
滑动窗口施密特触发边界检测、
颜色分割主策略 + 边缘精定位辅助策略双融合定位。
经多场景实测验证，本项目 FPGA 的 roi 模块框选成功效果：
纯车牌无干扰场景识别准确率达 97%（130/134）、
有背景 ROI 场景识别准确率达 86%（97/112）、
不同角度与光照变化 ROI 场景识别准确率达 81%（57/70）；
同时 ARM 也有极高的单体识别率：

车牌类型	数量	正确数	准确率
单层蓝牌	4576	4388	95.89%
单层黄牌	3131	2874	91.79%
双层黄牌	1579	1394	88.28%
拖拉机绿牌	1532	1489	97.19%
新能源小型车	4612	4591	99.54%
新能源大型车	3108	3040	97.81%
黑色车牌	1462	1427	97.61%

目录

总体设计方案

- 需求分析与实现结果
- 硬件架构图
- 功能实现与算法设计
- 测试报告（性能和指标分析）

FPGA 架构设计

- 目标功能
- 算法架构
- 整体算法流程图
- ROI 兜底机制和纠错逻辑
 - 算法流程图
 - 代码
- 核心算法实现——颜色中心查找
 - 数据流动视图
 - 算法流程图
- 核心算法实现——边界查找
 - 数据流动视图
 - 算法流程图

ARM 架构设计

- 算法总览
- 核心模块设计
 - 图像预处理模块
 - LPRNet 模型部署与推理
 - 车牌颜色识别与类型判定
 - 字符串后处理
 - 双层车牌处理逻辑
- 软件架构与模块依赖

附录

总体设计方案

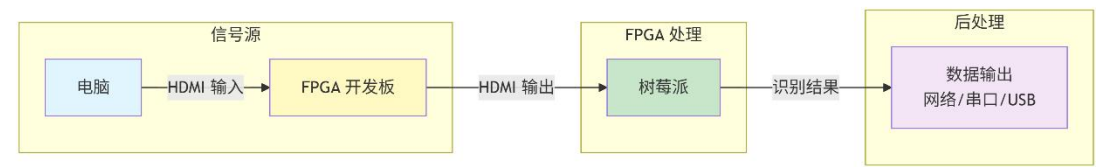
1.需求分析与实现效果

本项目面向嵌入式智能交通场景，基于紫光同创 **FPGA** 芯片，设计一套高性能，高鲁棒性，

高性价比的车牌识别系统，在评估要求和可行性后我们选择了 ARM+FPGA 方案，既保证了硬件处理图像的实时性，又有一定的性能兜底以提升识别正确率。成功达到了 HDMI 图像采集、处理、输出、识别的全链路功能需求，高度完成了预处理算法的实现并且经过调试有较强鲁棒性

2.硬件架构图

FPGA 开发板选择了紫光同创 PG2L100H，ARM 核上位机我们选择了拓展性和适配性极好的树莓派 5。数据传输/硬件架构如下：



3.功能实现与算法设计

简述算法设计 FPGA 层面: 通过 i2c 读取 ms72xx 显存芯片获取输入图片 RGB 图, 进行了 gamma 补偿, 灰度化, 高斯滤波, sobel 边缘检测, 形态学运算, 分别对 gamma 后的图像和二值化后的图像做了颜色向量中心识别和投影边缘检测, 加权最终的边界坐标得到 roi 框选。ARM 层面: 根据 FPGA 发来的带边框的图片中红色边框内的车牌进行识别

4.测试报告与识别性能

识别性能（数据来源于提交的视频所测，随机分类选取数据集）

测试维度	车牌类型/场景	FPGA实现ROI成功率	仅ARM识别成功率	合并成功率
按场景	纯车牌无干扰	蓝色97%(134样本) 绿色91%(96样本)		91%(134样本)
	有背景的车牌	蓝色86%(112样本) 绿色82%(56样本)		83%(112样本)
	不同光照的车牌	81%(79样本)		74%(79样本)
按类型	黑色车牌	~		89%(117样本)
	末尾带汉字	~		81%(49样本)
	拼色牌	95%(120样本)		54%(115样本)
	双层牌	~	95.65%(20000样本)	

仅 ARM 识别成功率

车牌类型	数量	正确数	准确率
单层蓝牌	4576	4388	95.89%
单层黄牌	3131	2874	91.79%
双层黄牌	1579	1394	88.28%
拖拉机绿牌	1532	1489	97.19%
新能源小型车	4612	4591	99.54%
新能源大型车	3108	3040	97.81%
黑色车牌	1462	1427	97.61%

测试报告

功能仿真验证方式：我们采用 ai 将 FPGA 上的算法转化为 cv，便于我们在更完善的 cv 库基础上寻找思路和进行调试。

识别性能数据分析：从 FPGA 角度来说，其 ROI 算法鲁棒性良好，对不同条件的车牌均有很高的框选率，对于有背景干扰和纯车牌在合并后的下降程度属于可接受范围。但是在极端光照环境下，FPGA 框选成功率大受影响，这主要是传统 gamma 补偿算法能力不够强大，将上限压低导致合并识别率降低。但是对于几个特殊车牌，由于只有车牌，FPGA 几乎没有什么压力，将图像送至 ARM 后，可以很好的识别到对比度大的黑牌和特征明显的汉字牌，但是由于我们对低功耗追求在 FPGA 和 ARM 同时忽略了对拼接牌的特殊适配，ARM 的性能没有冗余的单独为这种车牌做颜色分割，导致拼接牌识别率低，这也是后面主要优化方向。

资源占用和功耗分析：FPGA 资源占用极低，LUT 占用仅 14%、FF 占用仅 8%、BRAM 占用远低于 15%，预留超 70% 的逻辑资源余量
ARM 我们通过降频压缩功耗到 5w 以下，同时保留了最低可运行识别条件，完成了最低成本的部署

FPGA 算法架构和核心代码

1. 目标功能：HDMI 输入输出，图像预处理，ROI 框选

2. 算法架构：

采用紫光官方 IP 核实现对显存芯片的读取

对读取图像进行 RGB 三通道 gamma 补偿

补偿完成后进行灰度化

Sobel 算子通过梯度进行边缘检测

膨胀一次，腐蚀一次，用于去噪和保留车牌完整性

颜色识别模块（核心+自研）

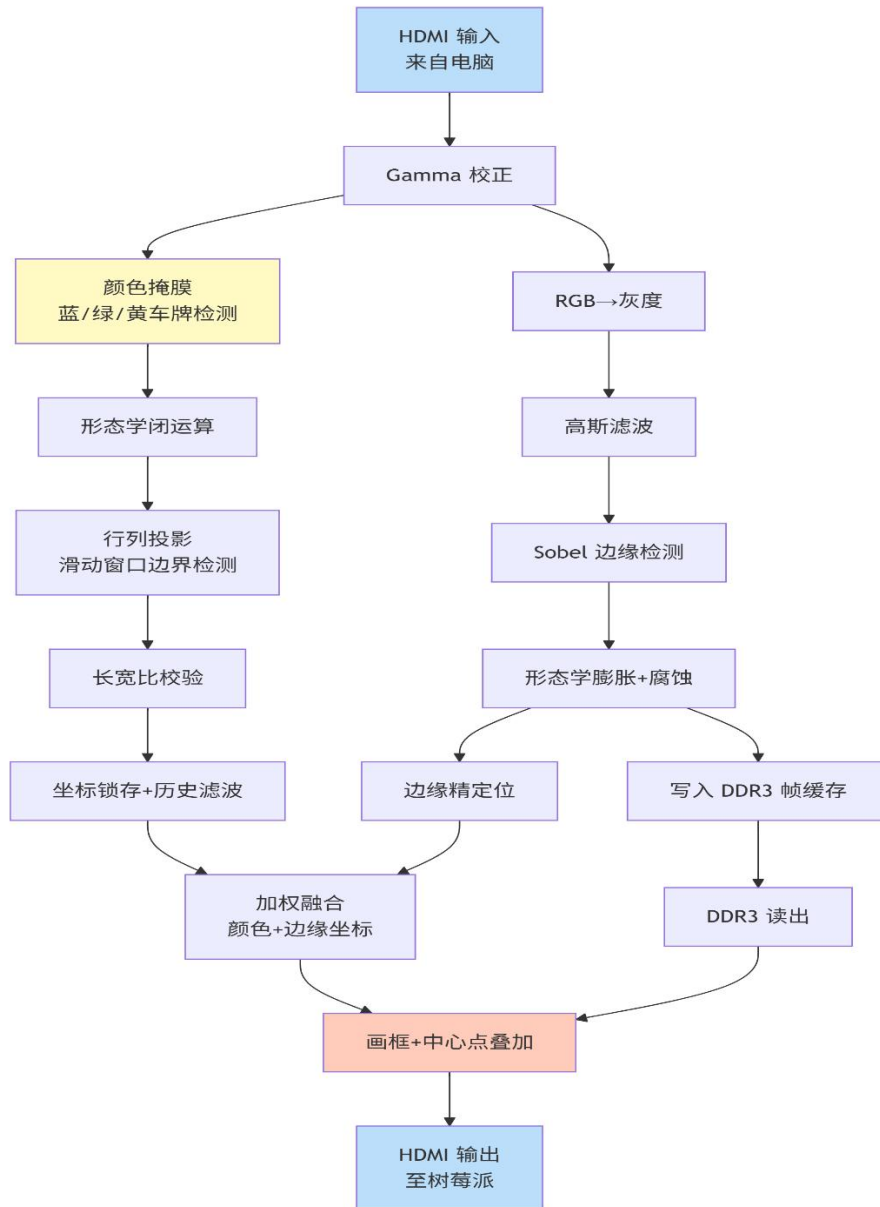
边界检查模块（核心+自研）

ROI 兜底机制：对中心检查和边界检查的数据进行加权，并根据情况判定框是否正确

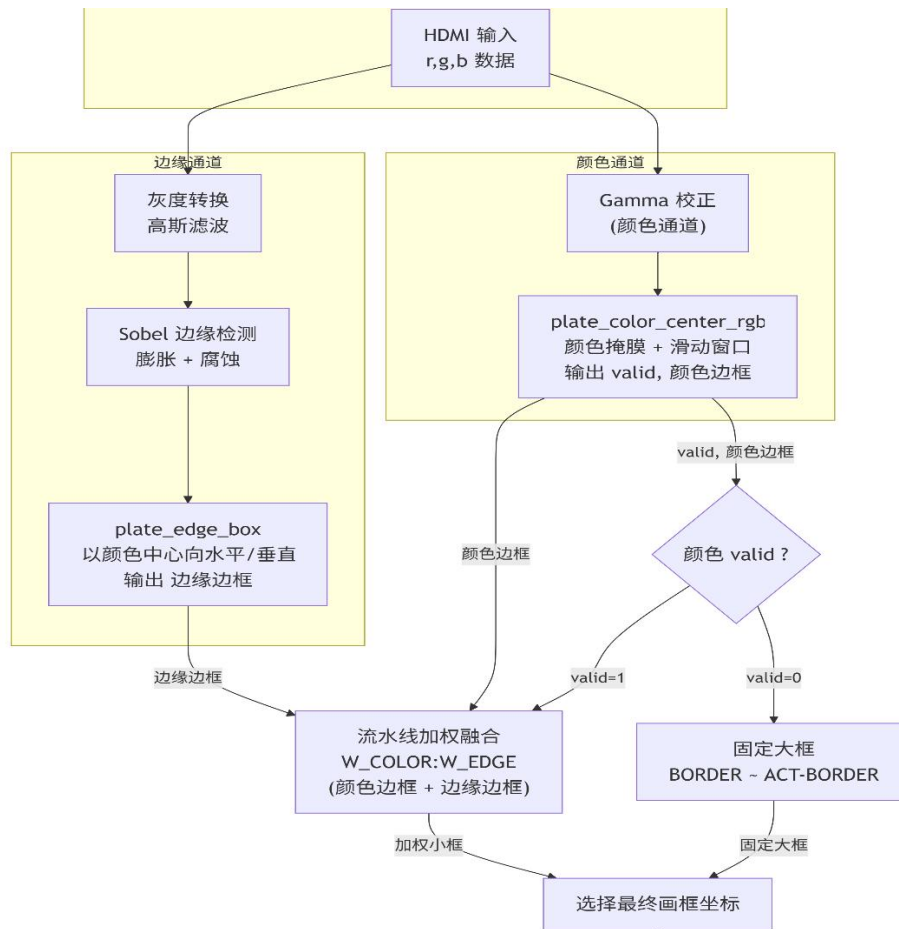
得到画框边界后送到画框的例化

叠加到显存后输出

3. 算法流程图



4. 关于 ROI 兜底机制的实现及纠错逻辑： 我们需要先把两个核心算法合在一起看，gamma 值输入 color_find 之后，其输出包含一个标志位 valid，当这个 valid 为 1，证明识别到正确的颜色中心了，同时下一个边缘寻找的算法也会返回其以颜色中心向水平竖直方向投影找到的边框坐标值，边缘寻找的执行与否与 valid 不直接挂钩。因此无论如何，边界寻找都会因其宽松的判定生成一个较大的框且居于中央，即使颜色寻找失败也不会大幅度影响框选，只是框住的噪声变多，但也减轻了上位机的负担。如果颜色寻找有效，考虑到腐蚀会造成车牌变小，所以我们同步时序之后对边框坐标进行加权（附图），让边框在原来的基础上适当变大保留完整车牌。这样的设计既保留了颜色寻找和比值筛选的相对准确性，也为特殊情况留下容错，纠错机制闭环。



```

parameter W_COLOR = 9; // 颜色权重
parameter W_EDGE = 1; // 边缘权重
localparam W_TOTAL = W_COLOR + W_EDGE;

// 同步后的坐标
wire [11:0] color_x1_final = color_x1_sync2;
wire [11:0] color_y1_final = color_y1_sync2;
wire [11:0] color_x2_final = color_x2_sync2;
wire [11:0] color_y2_final = color_y2_sync2;
wire [11:0] edge_x1_final = edge_x1_sync2;
wire [11:0] edge_y1_final = edge_y1_sync2;
wire [11:0] edge_x2_final = edge_x2_sync2;
wire [11:0] edge_y2_final = edge_y2_sync2;

reg [23:0] prod_x1, prod_y1, prod_x2, prod_y2;
reg prod_valid1, prod_valid2;

always @(posedge pix_clk) begin
    prod_x1 <= color_x1_final * W_COLOR + edge_x1_final * W_EDGE;
    prod_y1 <= color_y1_final * W_COLOR + edge_y1_final * W_EDGE;
    prod_x2 <= color_x2_final * W_COLOR + edge_x2_final * W_EDGE;
    prod_y2 <= color_y2_final * W_COLOR + edge_y2_final * W_EDGE;
    prod_valid1 <= center_vld_sync2;
    prod_valid2 <= prod_valid1;
end

reg [11:0] weighted_x1, weighted_y1, weighted_x2, weighted_y2;
reg draw_valid;

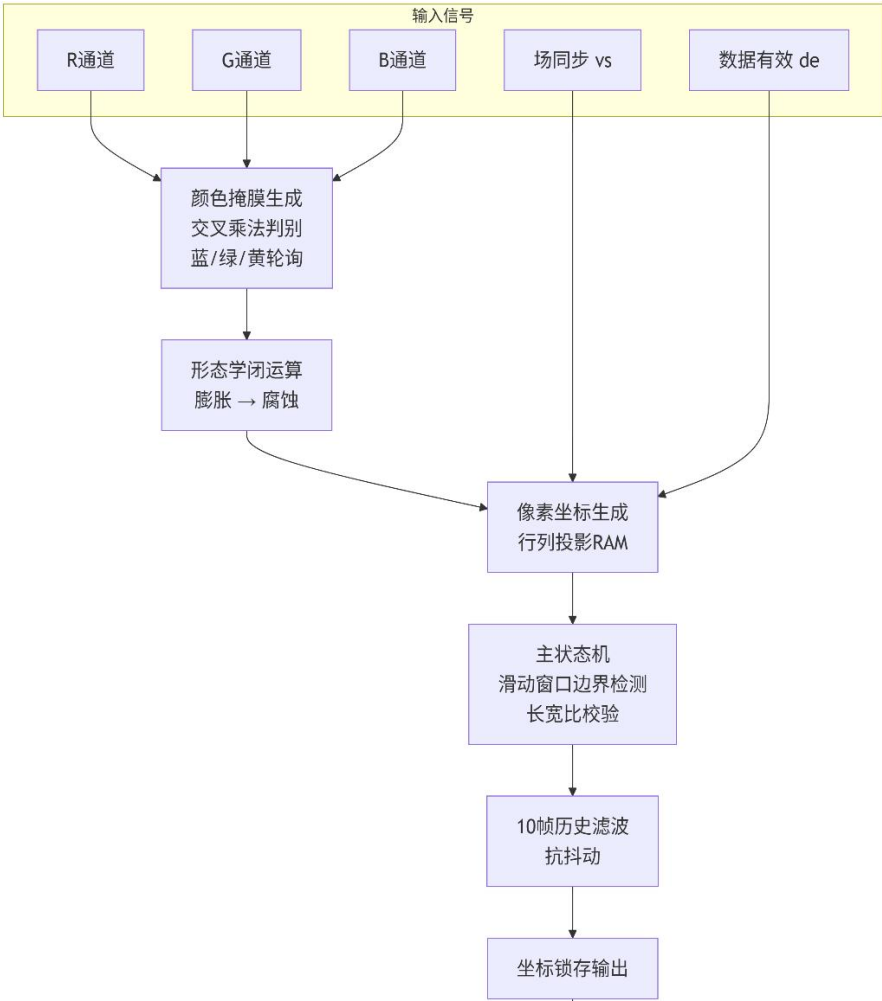
always @(posedge pix_clk) begin
    weighted_x1 <= prod_x1 / W_TOTAL;
    weighted_y1 <= prod_y1 / W_TOTAL;
    weighted_x2 <= prod_x2 / W_TOTAL;
    weighted_y2 <= prod_y2 / W_TOTAL;
    draw_valid <= prod_valid2;
end

```

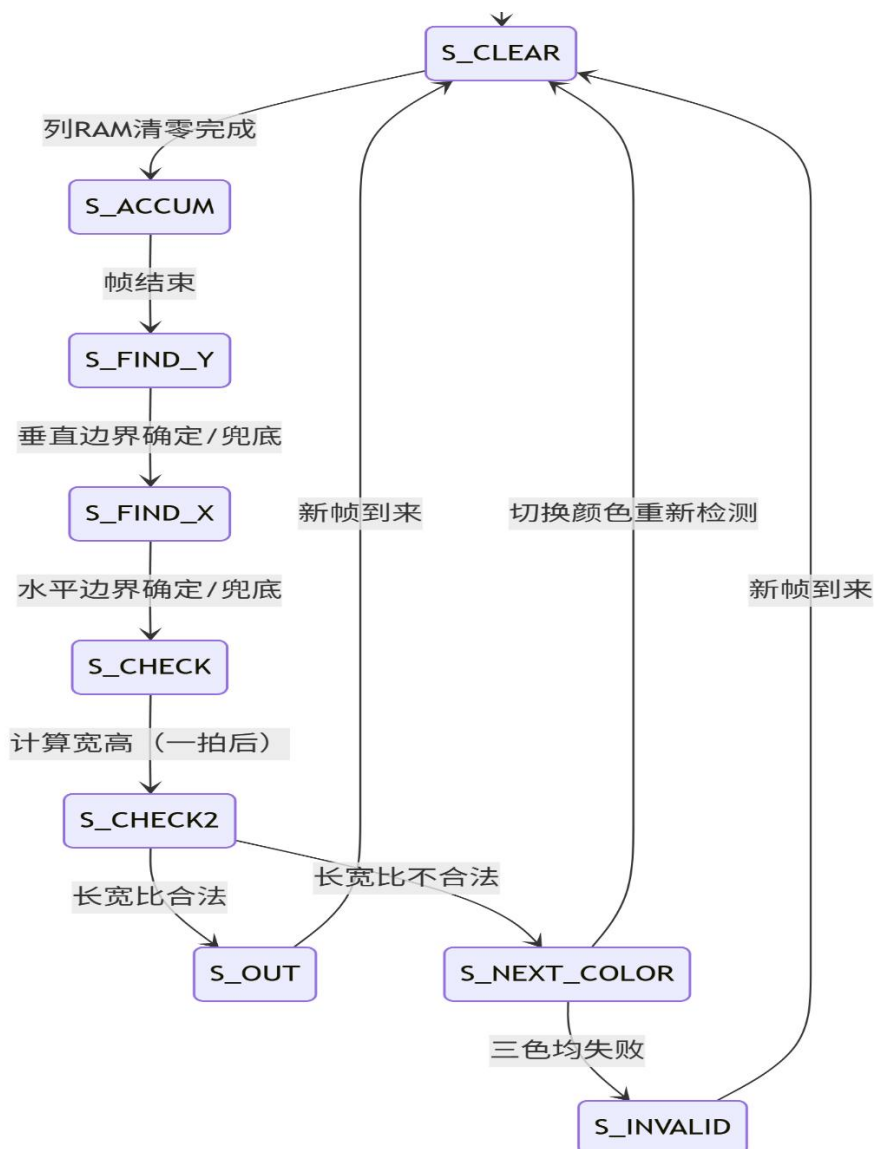

5. 核心算法实现——颜色中心识别算法（带注释代码见附件）

- 1. RGB 掩膜提取：（核心环节）通过通道比例关系精准筛选符合车牌颜色的像素，同时用乘法替代除法，完全规避浮点计算与精度误差，直接在 RGB 原始色彩空间完成，无需转换 HSV 等其他颜色空间，每次只生成一种颜色的掩膜，防止相互干扰。
- 2. 形态学运算处理掩膜
- 3. 投影查找颜色边界：将二维的二值图转换为两组一维统计数组，把复杂的二维边界查找，简化为一维数组的峰值区间定位，同时采用固定大小的滑动窗口，在投影数组上逐位滑动，计算窗口内的数值总和，消除数组中的小幅波动与噪声，让峰值区间更平滑。窗口总和超过高阈值时，判定进入车牌区域，记录当前位置为起始边界（上边界/左边界），窗口总和低于低阈值时，判定离开车牌区域，记录当前位置为结束边界（下边界/右边界）
- 4. 比例检查：根据左右边界值算矩形比例是否符合车牌比例，根据这个生成 valid
- 5. 加入多重颜色轮询机制
- 6. 滑动窗口滤波：用长度为 10 的滑动窗口，记录连续 10 帧图像的检测结果，统计窗口内的有效帧数量，当有效帧 ≥ 7 帧时，才输出最终的有效检测标志，有效减少框选闪烁

数据流动视图



状态机逻辑



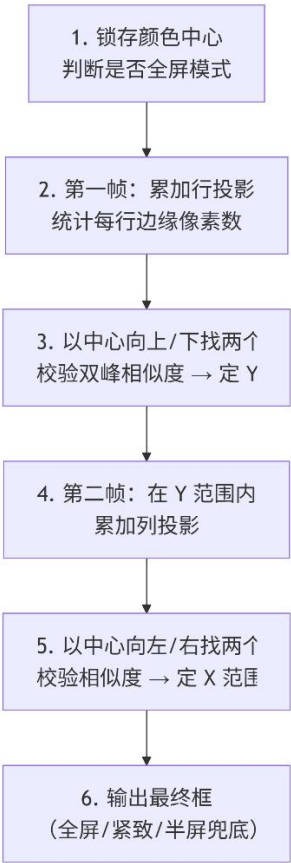
5. 核心算法实现——边界寻找（带注释代码见附件）

1. **颜色中心驱动与双帧串行投影**：模块不独立搜索全图，而是以颜色检测模块输出的车牌中心坐标 (cx, cy) 为锚点，分两帧依次完成 Y 方向和 X 方向的边界精定位。第一帧统计行投影（每行边缘像素总数），用于确定垂直边界；第二帧在已确定的垂直范围内统计列投影（每列边缘像素总数），用于确定水平边界。这种“先 Y 后 X”的两帧流水线设计既节省了存储资源（只需一个行 RAM 和一个列 RAM），又让投影数据严格限定在车牌附近区域，大幅抑制背景噪声。

2. **双向峰搜索与双峰相似度校验**：以颜色中心为出发点，分别向上、下（或左、右）两个方向扫描投影数组，各自记录最大值所在的“峰”位置和峰高。得到双峰后，通过公式 $(\text{小峰} \times 100) \geq (\text{大峰} \times \text{SIM_THRESH})$ 检查两峰是否“高度相似”（默认阈值 85%）。只有双峰都真实存在且强度匹配，才判定为有效车牌边界，否则认为边缘信息不可靠，触发兜底策略。

3. **全屏兜底与半屏兜底，保证永不空框**：如果输入的颜色中心为 $(0, 0)$ （即颜色检测完全失效），模块直接输出覆盖整个有效画面的全屏大框，相当于“找不到车牌就把整个有效区域交给上位机”，彻底杜绝因颜色失效导致的漏检。若颜色中心有效但峰搜索失败（例如边缘图破碎、双峰不对称），则采用以颜色中心为中心、半屏宽/高为半径的矩形作为保守边界，确保在边缘信息不理想时仍能输出一个包含车牌的合理范围，极大地提高了极端工况下的鲁棒性。

4. 最终框选输出和模式切换：状态机最终在 S_OUTPUT 状态根据颜色中心有效性 (full_screen 和峰搜索结果，选择以下三种之一输出最终边框：颜色失效 → 全屏；(0, 0) ~ (H_ACT-1, V_ACT-1) 颜色有效 + 峰对找到 → 峰最小/最大位置构成的紧致框；颜色有效 + 峰对未找到 → 以颜色中心为中心的半屏兜底框。所有输出直接连接到顶层加权融合模块，构成完整的“颜色+边缘”双路冗余架构。



ARM 算法架构和核心代码

1. 算法总览

ARM 端使用树莓派 5，未搭载 AI Kit，无 NPU 与 GPU 算力，系统为 Ubuntu24.04。ARM 端接收来自 FPGA 的带红色 ROI 边框的 HDMI 图像，通过裁剪边框区域获取车牌局部图像，送入识别流水线。流水线由图像预处理、LPRNet 深度学习字符识别、字符后处理与车牌类型判定四个核心环节组成，最终输出车牌字符串与车辆类型。

与 FPGA 的纯硬件流水线分工明确：

- 1、FPGA 负责：**全图级图像增强、颜色定位、边缘精定位、ROI 框选与叠加。
- 2、ARM 负责：**ROI 裁剪、车牌颜色判断、透视校正、双层车牌处理、LPRNet 字符识别、字符串清洗与类型判定。

2. 核心模块设计

2.1 图像预处理模块

ARM 端通过 HDMI 转 USB 接收到 FPGA 输出图像后，首先进行 ROI 裁剪，获取红色边框内的车牌区域图像。

预处理流程包括：

- ①Gamma 校正：增强暗部细节，提升夜间及逆光场景下字符区域对比度。
- ②透视变换：通过最小外接矩形拟合车牌轮廓，将倾斜车牌矫正为正向矩形，降低后续 LPRNet 识别难度。（效果不好，未启用）
- ③尺寸归一化：将校正后的车牌图像缩放至 LPRNet 网络输入尺寸（94×24 像素），保持宽高比填充。

2.2 LPRNet 模型部署与推理

字符识别采用轻量级 LPRNet 网络，模型结构以卷积层堆叠 + small basic block 为主干，通过 MaxPool3d 降维后连接全局平均池化与 CTC 输出头，直接输出不定长字符序列的概率分布，无需字符分割，模型如下：

Input	in0	0 1 in0
Convolution	convrelu_0	1 1 in0 1 0=64 1=3 5=1 6=576 9=1
Split	splitcnnn_0	1 2 1 2 3
Reshape	reshape_45	1 1 1 4 0=92 1=22 11=64 2=1
Pooling3D	maxpool3d_27	1 1 4 5 1=3 21=1 6=1
Reshape	reshape_46	1 1 5 6 0=90 1=20 2=64
Split	splitcnnn_1	1 2 6 7 8
Convolution	convrelu_1	1 1 8 9 0=32 1=1 5=1 6=2048 9=1
Convolution	convrelu_2	1 1 9 10 0=32 1=1 11=3 14=1 5=1 6=3072 9=1
Convolution	convrelu_3	1 1 10 11 0=32 1=3 11=1 4=1 14=0 5=1 6=3072 9=1
Convolution	conv_13	1 1 1 11 12 0=128 1=1 5=1 6=4096
Convolution	conv_14	1 1 7 13 0=128 1=1 5=1 6=8192
BinaryOp	add_0	2 1 12 13 14
ReLU	relu_3	1 1 14 15
BatchNorm	bn_6	1 1 15 16 0=128 1=1.000000e-05
ReLU	relu_34	1 1 16 17
Split	splitcnnn_2	1 2 17 18 19
Reshape	reshape_47	1 1 19 20 0=90 1=20 11=128 2=1
Pooling3D	maxpool3d_28	1 1 20 21 1=3 21=1 2=2 12=1 5=1
Reshape	reshape_48	1 1 21 22 0=44 1=18 2=64
Split	splitcnnn_3	1 2 22 23 24
Convolution	convrelu_4	1 1 24 25 0=64 1=1 5=1 6=4096 9=1
Convolution	convrelu_5	1 1 25 26 0=64 1=1 11=3 14=1 5=1 6=12288 9=1
Convolution	convrelu_6	1 1 26 27 0=64 1=3 11=1 4=1 14=0 5=1 6=12288 9=1
Convolution	conv_18	1 1 27 28 0=256 1=1 5=1 6=16384
Convolution	conv_19	1 1 23 29 0=256 1=1 5=1 6=16384
BinaryOp	add_1	2 1 28 29 30
ReLU	relu_4	1 1 30 31
BatchNorm	bn_7	1 1 31 32 0=256 1=1.000000e-05
ReLU	relu_38	1 1 32 33
Split	splitcnnn_4	1 2 33 34 35
Convolution	convrelu_7	1 1 35 36 0=64 1=1 5=1 6=16384 9=1
Convolution	convrelu_8	1 1 36 37 0=64 1=1 11=3 14=1 5=1 6=12288 9=1
Convolution	convrelu_9	1 1 37 38 0=64 1=3 11=1 4=1 14=0 5=1 6=12288 9=1
Convolution	conv_23	1 1 38 39 0=256 1=1 5=1 6=16384
BinaryOp	add_2	2 1 39 34 40
ReLU	relu_5	1 1 40 41
BatchNorm	bn_8	1 1 41 42 0=256 1=1.000000e-05
ReLU	relu_42	1 1 42 43
Split	splitcnnn_5	1 2 43 44 45
Reshape	reshape_49	1 1 45 46 0=44 1=18 11=256 2=1
Pooling3D	maxpool3d_29	1 1 46 47 1=3 21=1 2=2 12=1 22=4 5=1
Reshape	reshape_50	1 1 47 48 0=21 1=16 2=64
Convolution	convrelu_10	1 1 48 49 0=256 1=4 11=1 5=1 6=65536 9=1
Convolution	convrelu_11	1 1 49 50 0=76 1=1 11=13 5=1 6=252928 9=1
Split	splitcnnn_6	1 2 50 51 52
Pooling	avgpool2d_0	1 1 2 53 0=1 1=5 2=5 5=1 6=1
Split	splitcnnn_7	1 2 53 54 55
UnaryOp	pow_3	1 1 54 56 0=4
Reduction	mean_52	1 1 56 57 0=3
BinaryOp	add_4	1 1 57 58 1=1 2=1.000000e-08
BinaryOp	div_5	2 1 55 58 59 0=3
Pooling	avgpool2d_1	1 1 18 60 0=1 1=5 2=5 5=1 6=1
Split	splitcnnn_8	1 2 60 61 62
UnaryOp	pow_6	1 1 61 63 0=4
Reduction	mean_53	1 1 63 64 0=3
BinaryOp	add_7	1 1 64 65 1=1 2=1.000000e-08
BinaryOp	div_8	2 1 62 65 66 0=3
Pooling	avgpool2d_2	1 1 44 67 0=1 1=10 11=4 2=2 12=4 5=1 6=1
Split	splitcnnn_9	1 2 67 68 69
UnaryOp	pow_9	1 1 68 70 0=4
Reduction	mean_54	1 1 70 71 0=3
BinaryOp	add_10	1 1 71 72 1=1 2=1.000000e-08
BinaryOp	div_11	2 1 69 72 73 0=3
UnaryOp	pow_12	1 1 51 74 0=4
Reduction	mean_55	1 1 74 75 0=3
BinaryOp	add_13	1 1 75 76 1=1 2=1.000000e-08
BinaryOp	div_14	2 1 52 76 77 0=3
Concat	cat_0	4 1 59 66 73 77 78
Convolution	conv_26	1 1 78 79 0=76 1=1 5=1 6=39824
Reduction	mean_51	1 1 79 out0 0=3 1=0 -23303=1,1 5=1

模型权重文件 lprnet_double.pth 已针对本项目应用场景在 CBLPRD 数据集上进行训练，支持以下字符集：

- ①省份简称：皖、沪、津、渝、冀、晋、蒙、辽、吉、黑、苏、浙、京、闽、赣、鲁、豫、鄂、湘、粤、桂、琼、川、贵、云、藏、陕、甘、青、宁、新（31 个）
- ②字母数字：A~Z、0~9（36 个）
- ③特殊字符：学、警、港、澳、挂、使、领、临（8 个）

训练集中双层车牌数据进行了上下切割、左右拼接，以弥补 LPRNet 只适合识别单行文字的问题，对比未对双层车牌进行特殊处理，在 CBLPRD 数据集上的总体正确率从 80%提升至最高 97.8%。后端支持 PyTorch 与 NCNN 两种模式。本次部署在树莓派上采用 NCNN 推理，单帧识别延迟低于 250 ms，满足嵌入式实时识别需求。

2.3 车牌颜色识别与类型判定

颜色识别在 ARM 端独立完成，用于辅助判定车牌种类。具体流程：

- ①将 ROI 图像从 RGB 转换至 HSV 色彩空间。
- ②基于 H、S、V 通道范围分区判断颜色类别：蓝、黄、绿、白、黑。
- ③结合颜色类别与字符识别结果，判定最终车牌类型。

颜色判定为后续字符串清洗和字符集约束提供了关键先验信息。

2.4 字符串后处理

LPRNet 输出的原始字符序列与类型判断不一定准确，故我们基于车牌的规律对一些特殊车牌做了一些逻辑判断进行纠错：

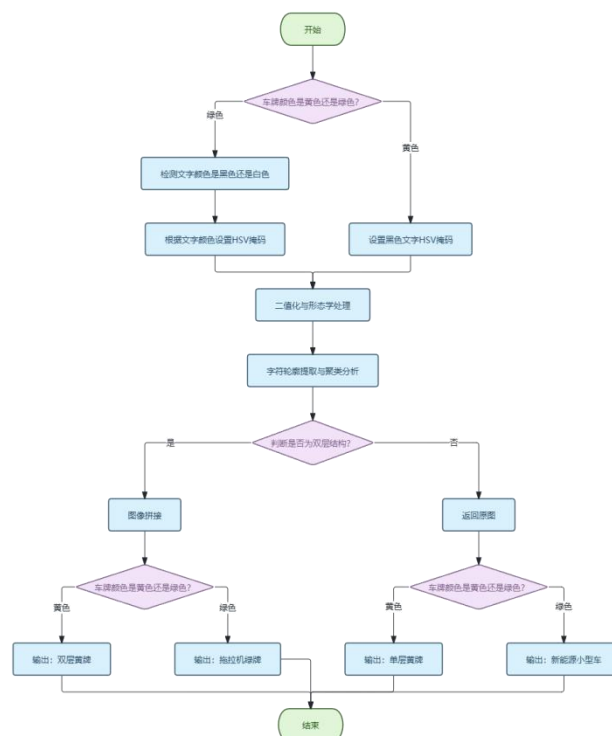
- ①若最后一个字是“使”，则去掉最前面可能出现的省份；
- ②若最后一个字是“警”，“港”，“澳”，“使”，“领”，则车牌必为黑色
- ③若最后一个字是“临”，则车牌必为拖拉机绿牌
- ④若最后一个字是“挂”，则车牌必为双层黄牌
- ⑤去掉识别结果中多于 8 个字的部分

2.5 双层车牌处理逻辑

针对双层黄牌，ARM 端实现了专门的双层检测与拼接算法：

- ①字符行聚类：对车牌二值化后的字符连通域进行 Y 坐标聚类，判断是否存在两行字符。
- ②上下行拼接：检测到双行结构后，将上下两行字符区域分别展开并拼接为单行图像，再送入 LPRNet 进行统一识别。

该模块补足了 LPRNet 模型只适合识别单行文字的问题，提升了对特殊结构车牌的适应能力。



2.6 行人识别

通过空格切换识别车牌与识别行人，行人识别部分采用 yolov26n 模型，考虑到 yolov26 对嵌入式设备的优化，本项目大胆地采用了该较为新的模型，并转化为 ncnn 模型并进行 int8 量化，进一步提升其性能。

Yolo 默认模型对人的识别正确率极高，本项目未对其进行正确率测试，可参考官方数据。

3. 软件架构

